
```
In [1]: from __future__ import print_function, division
import numpy as np
import matplotlib as mpl
import matplotlib.pyplot as plt
%matplotlib inline
plt.rc('figure', figsize=(8.0, 4.0), dpi=140)
```



Progressive Growing of GANs for Improved Quality, Stability, and Variation

Cool video



tanh relu sigmoid

```
In [2]: # here is a neural network

# compute the prediction (output) of a
# deep neural network
# given a list of weight matrices

def G(x):
```

```

    return np.maximum(x, 0) # ReLU

def predict(x, weights, biases):
    for b_i, w_i in zip(biases, weights):
        x = G(w_i @ x + b_i)
    return x

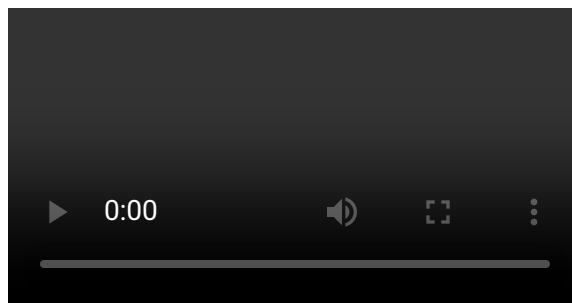
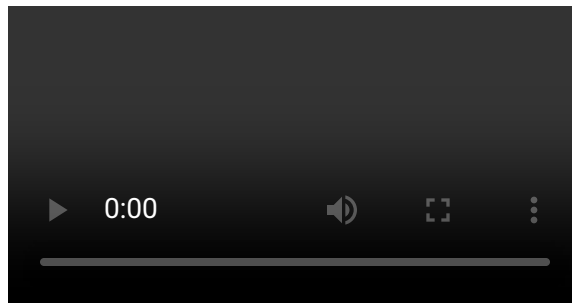
# "all" we have to do is adjust "weights" and
# "biases" until we get the result we want :)
# These are our parameter vector, theta

def train(x, y, w_shapes, b_shapes):
    #???
    pass

```

[colah's blog](#)
[another colah article](#)
[Nielsen's free book](#)
[Deep learning from scratch](#)





```
In [3]: f = lambda x:x**4 - 40*x**2 - 100*x  
df = lambda x:4*x**3 - 80*x - 100  
ddf = lambda x:12*x**2-80
```

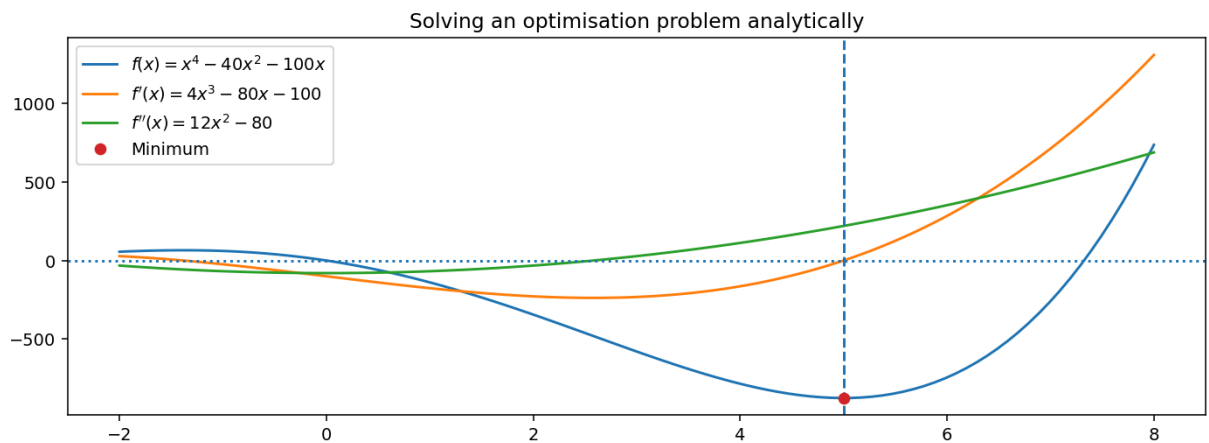
```

x = np.linspace(-2,8,100)
fig = plt.figure(figsize=(12, 4))
ax = fig.add_subplot(1,1,1)
ax.plot(x, f(x), label="$f(x)=x^4 - 40x^2 - 100x$")
ax.plot(x, df(x), label="$f'\text{prime}(x)=4x^3 - 80x - 100$")
ax.plot(x, ddf(x), label="$f'\text{prime}\text{prime}(x)= 12x^2 - 80$")

# solution is 5.0 by inspection
ax.axhline(0, ls=':')
ax.axvline(5.0, ls='--')
ax.plot(5.0, f(5.0), 'o', label='Minimum')
ax.legend()
ax.set_title("Solving an optimisation problem analytically")
print(df(5.0), ddf(5.0))
plt.show()

```

0.0 220.0



```

# vector -> scalar
def objective(theta):
    ...
    return score

# vector -> vector
def grad_objective(theta):
    ...
    return score_gradient

```



```
In [38]: import utils.history
import importlib as imp; imp.reload(utils.history)
from utils.history import History

#
# note: fixed step size isn't a good idea!

def gradient_descent(L, dL,
                    theta_0,
                    delta, tol=1e-4):
    """
    L: scalar loss function
    dL: gradient of loss function w.r.t parameters
    theta_0: starting point
    delta: step size
    tol: termination condition;
        when change in loss is less than tol, stop iterating
    """
    theta = np.array(theta_0) # copy theta_0
    o = History()
    o.track(np.array(theta), L(theta))

    # while the loss changes
    while np.sum(np.abs(dL(theta)))>tol:
        # step along the derivative
        theta += -delta * dL(theta)
        o.track(np.array(theta), L(theta))

    return o.finalise()
```

```
In [5]: def L(theta):
        return theta**2

def dL(theta):
    # we can differentiate L(theta) in our heads :)
    return 2*theta
```

```
In [6]: def plot_gradient_descent(xs, L, dL, x_0, step):
        # do descent
        res = gradient_descent(L, dL, x_0, step)

        # plot
        fig = plt.figure()
        ax = fig.add_subplot(1,1,1)
```

```

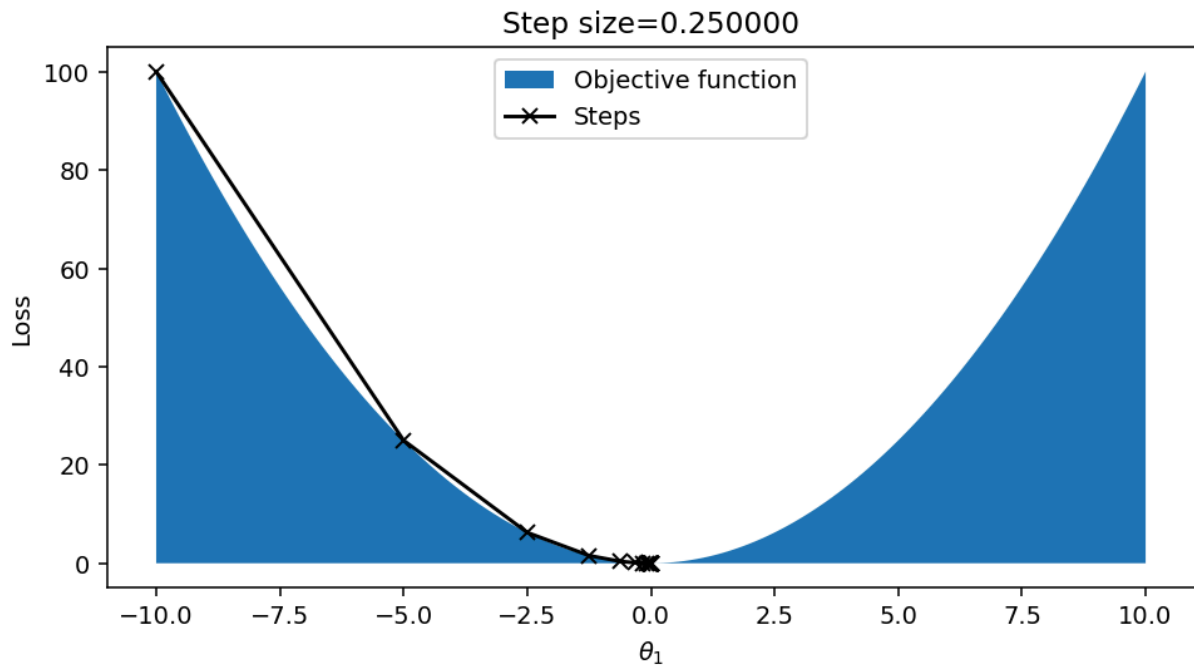
ax.fill_between(xs, L(xs), label="Objective function")
ax.set_title("Step size=%f" % step)
ax.set_xlabel("$\\theta_1$")
ax.set_ylabel("Loss")
ax.plot(res.best_thetas, res.best_losses, 'k-x', label='Steps')
ax.legend()
print("Converged in {0} steps".format(res.iters))
plt.show()

xs = np.linspace(-10,10,200)

```

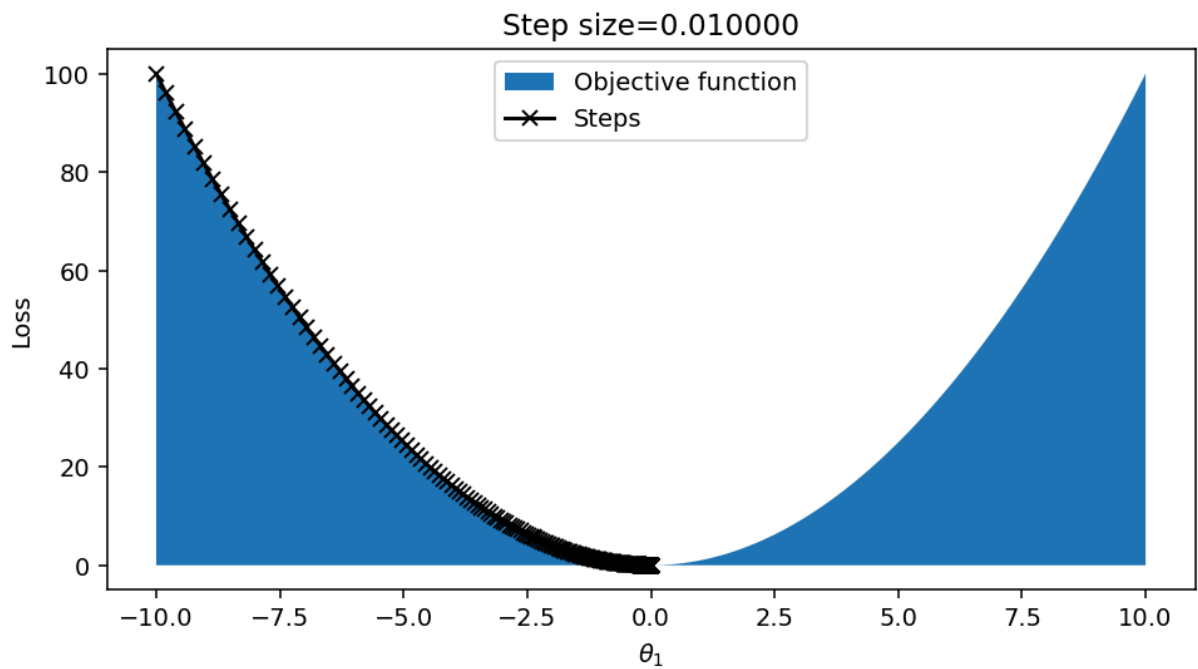
In [7]: `plot_gradient_descent(xs, L, dL, [-10.], 0.25)`

Converged in 19 steps



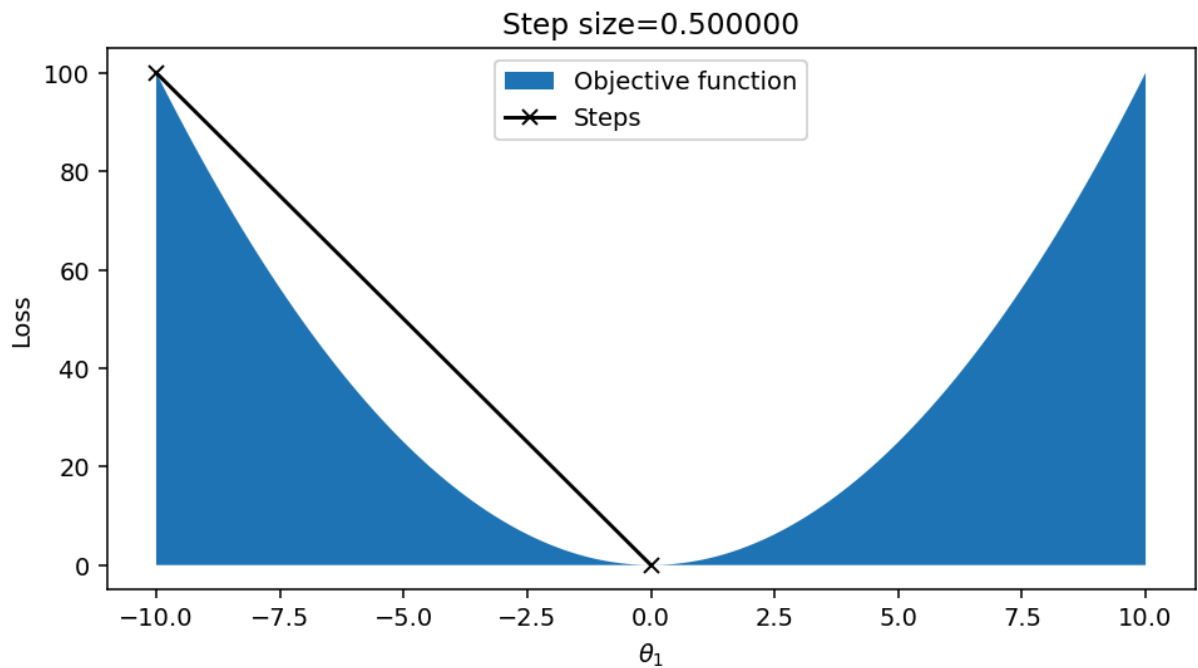
In [8]: `## too slow`
`# step = 0.01`
`plot_gradient_descent(xs, L, dL, [-10.], 0.01)`

Converged in 606 steps



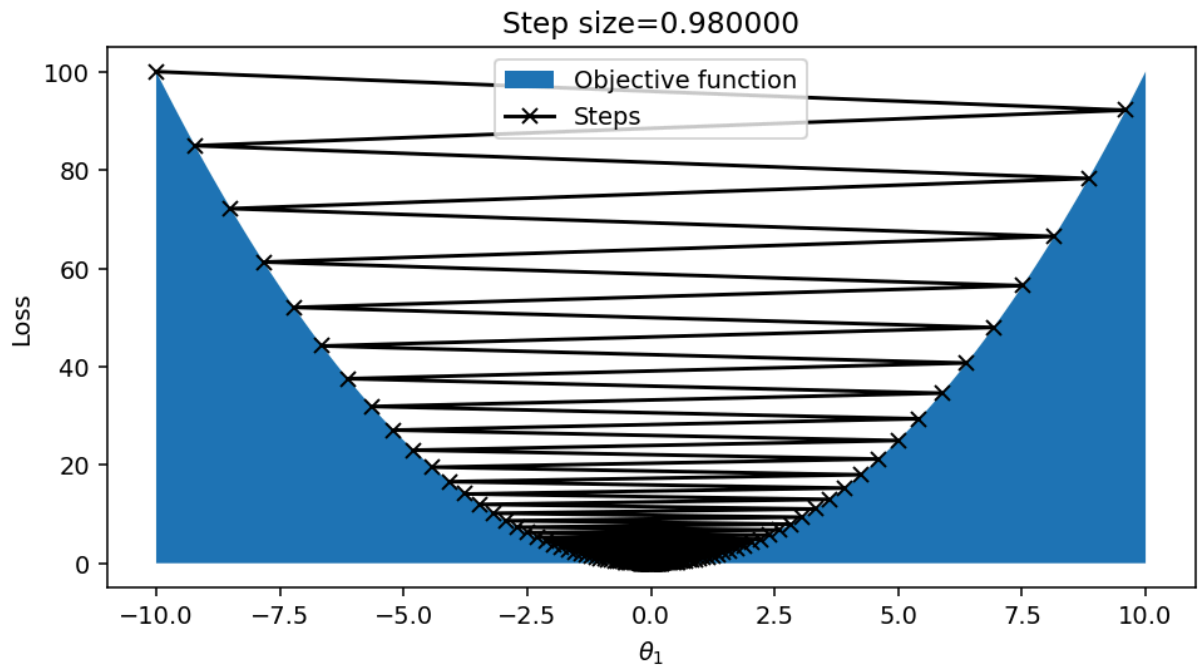
```
In [9]: # perfect :)
# step = 0.5
plot_gradient_descent(xs, L, dL, [-10.], 0.5)
```

Converged in 2 steps



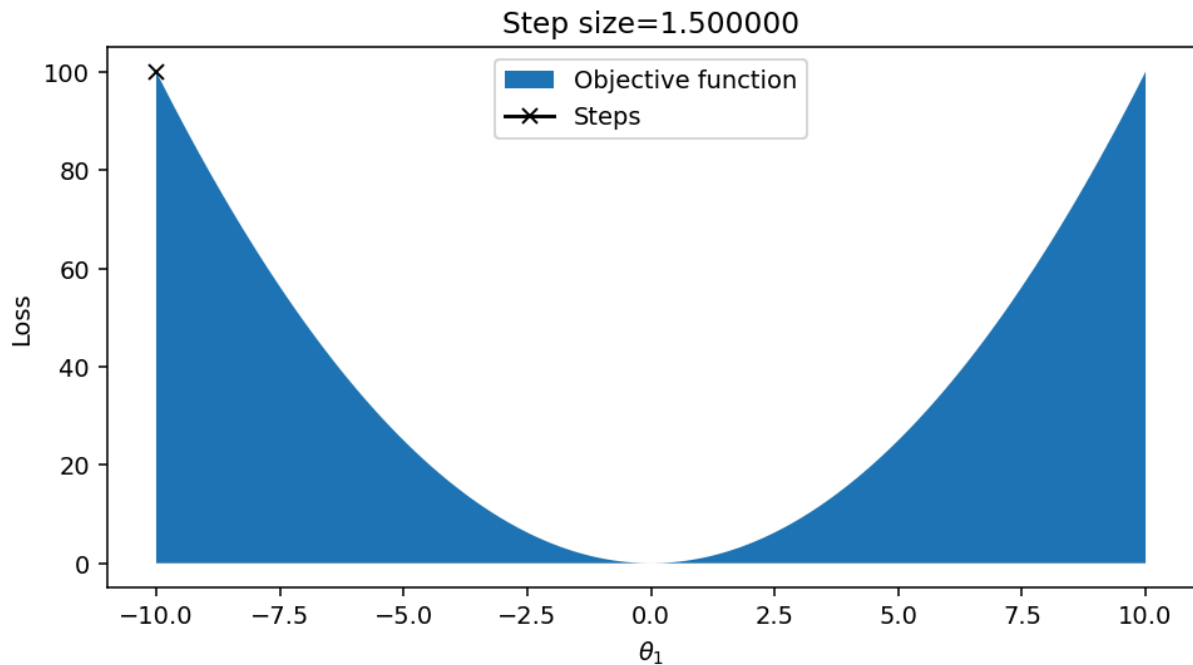
```
In [10]: # too fast -- oscillation occurs
plot_gradient_descent(xs, L, dL, [-10.], 0.98)
```

Converged in 301 steps



```
In [11]: # way too fast -- diverges
plot_gradient_descent(xs, L, dL,
                      [-10.], 1.5)
```

```
/tmp/ipykernel_5100/552418390.py:2: RuntimeWarning: overflow encountered in
square
    return theta**2
/mnt/c/Users/nicol/OneDrive - University of Glasgow/Lectures/2025-26/IDSS/Lectures/L5 - Optimisation II/utils/history.py:81: RuntimeWarning: invalid value encountered in subtract
    self.loss_change = self.last_loss - loss
/tmp/ipykernel_5100/552418390.py:6: RuntimeWarning: overflow encountered in multiply
    return 2*theta
/tmp/ipykernel_5100/1989288910.py:26: RuntimeWarning: invalid value encountered in add
    theta += -delta * dL(theta)
Converged in 1023 steps
```

```
In [12]: # test gradient descent in 2D
# you don't need to understand this code

### create a test function
from numpy.polynomial import polynomial

# a compact way of writing
#  $y^2 - 0.02y^3 + 5x + x^2 + 0.04x^3$ 
poly = np.array([[0,0,1,-0.02],
                  [5,0,0,0],
                  [1,0,0,0],
                  [0.04,0,0,0]])

# we can take the derivative exactly,
# since polynomial functions have known analytic derivative
# this function does this for us --
# it is a very simple array operation
der_x, der_y = (polynomial.polyder(poly, axis=0),
                 polynomial.polyder(poly, axis=1))

# loss
def L(x):
    return polynomial.polyval2d(x[0], x[1], poly)

# note that the gradient has two components,
# because its a 2D function
def dL(x):
    return np.array([polynomial.polyval2d(x[0], x[1], der_x),
                     polynomial.polyval2d(x[0], x[1], der_y)])
```

```

In [13]: # plot the surface
div = np.linspace(-35,35,100)
mx, my = np.meshgrid(div, div)
mz = L([mx, my])

fig = plt.figure()
ax = fig.add_subplot(1,1,1)
contour = ax.contourf(mx, my, mz, 32)
ax.contour(mx, my, mz, 30, colors='k', alpha=0.2)

# now plot some optimisation histories
# using gradient descent
# all of them with the same fixed size
res = gradient_descent(L, dL, [30.0,-40.0], 0.1, tol=1e-6)
history = res.best_thetas

ax.plot(history[:,0], history[:,1], 'C0-x')

res = gradient_descent(L, dL, [-10.0,20.0], 0.1, tol=1e-6)
history = res.best_thetas
ax.plot(history[:,0], history[:,1], 'C1-x')

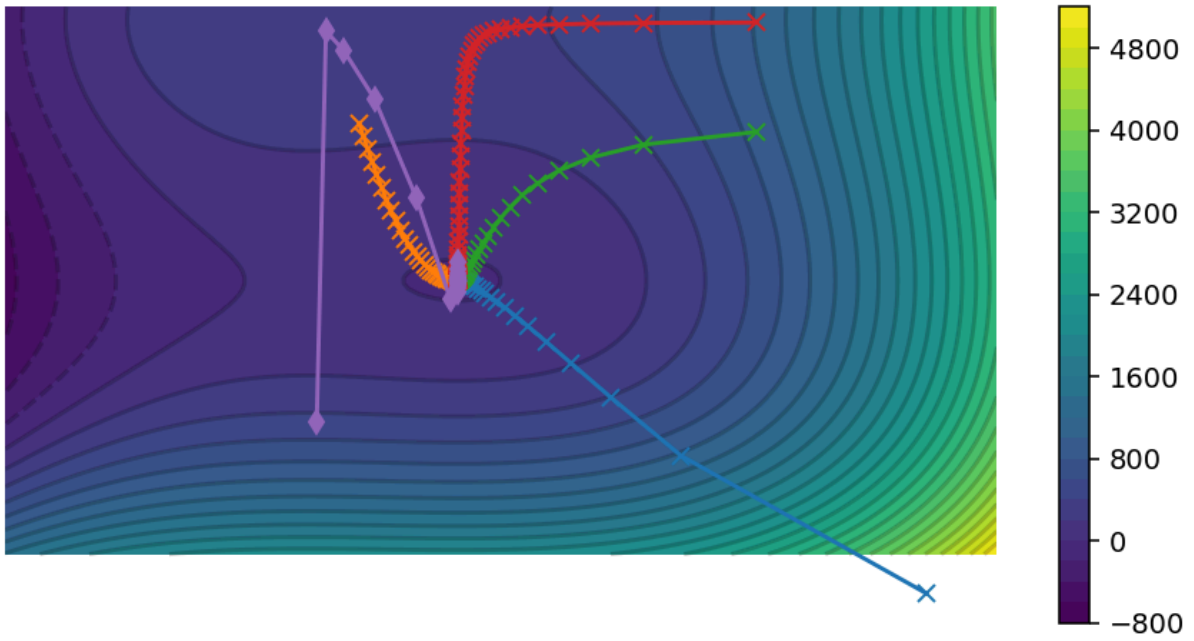
res = gradient_descent(L, dL, [18.0,19.0], 0.1, tol=1e-6)
history = res.best_thetas
ax.plot(history[:,0], history[:,1], 'C2-x')

res = gradient_descent(L, dL, [18.0,33.0], 0.1, tol=1e-6)
history = res.best_thetas
ax.plot(history[:,0], history[:,1], 'C3-x')

res = gradient_descent(L, dL, [-13.0,-18.0], 0.9, tol=1e-6)
history = res.best_thetas
ax.plot(history[:,0], history[:,1], 'C4-d')
ax.axis("off")
ax.set_title("Gradient descent from various starting positions")
fig.colorbar(contour)
plt.show()

```

Gradient descent from various starting positions



```
In [14]: def f(x):
    return x**(np.cos(x))

def df(x): # true, exact derivative
    return x**(np.cos(x)-1) * (np.cos(x)-x*np.log(x)*np.sin(x))

# finite differences
def approx_d(f, x, delta=1e-4):
    return (f(x+delta) - f(x-delta)) / (2*delta)

# plot the results over the range -10, 10
fig = plt.figure(figsize=(8,8))
ax = fig.add_subplot(2,1,1)
x = np.linspace(0,20, 100)

plt.rcParams["axes.prop_cycle"] = plt.cycler("color",
plt.cm.viridis(np.linspace(0,1,6)))

## Plot the derivative and its approximate form
ax.plot(x,df(x), label="True",lw=2)
for d in range(0,18,3):
    ax.plot(x,approx_d(f, x, delta=5**-d), ls='--', lw=1,
    label="$h=\\%.3e$"%(5**-d))

ax.legend(loc='right')
```

```

ax.set_title("Numerical differences versus true derivative of
 $x^{\cos(x)}$ ")
ax.set_xlabel("x")
ax.set_ylabel(" $f'(x)$ ")

## Plot the residuals
ax = fig.add_subplot(2,1,2)

for d in range(0,20,3):
    # adjust to 20 to see numerical issues
    d = d
    ax.plot(x,np.abs(df(x)-approx_d(f, x, delta=5**-d)), ls='-', lw=1,
label=" $h=0.3e^{5^{-d}}$ ")

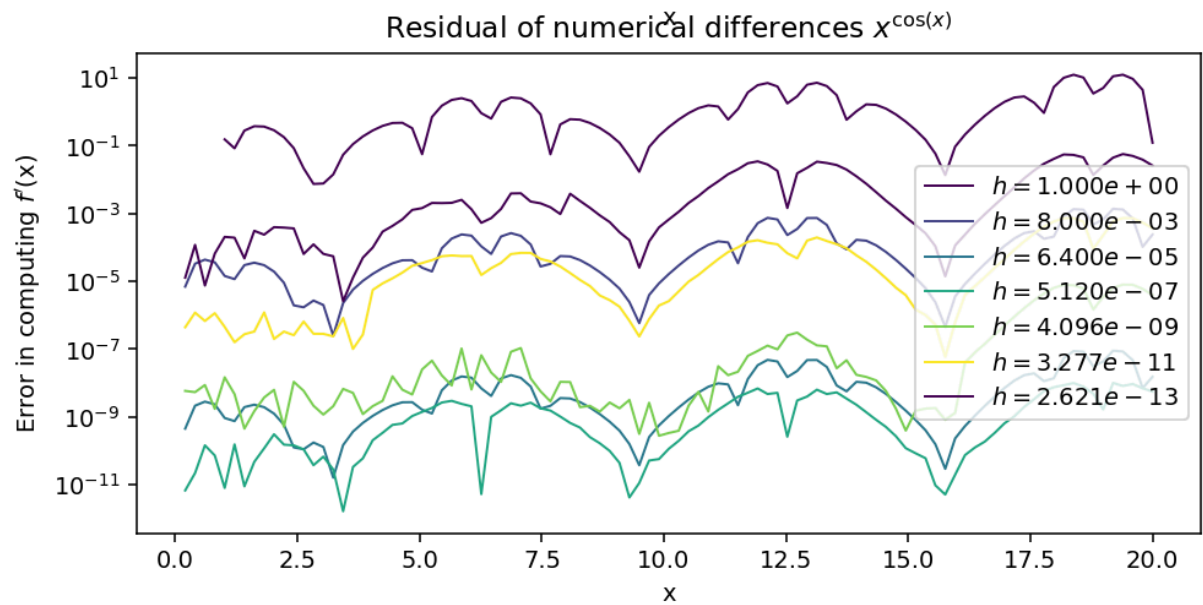
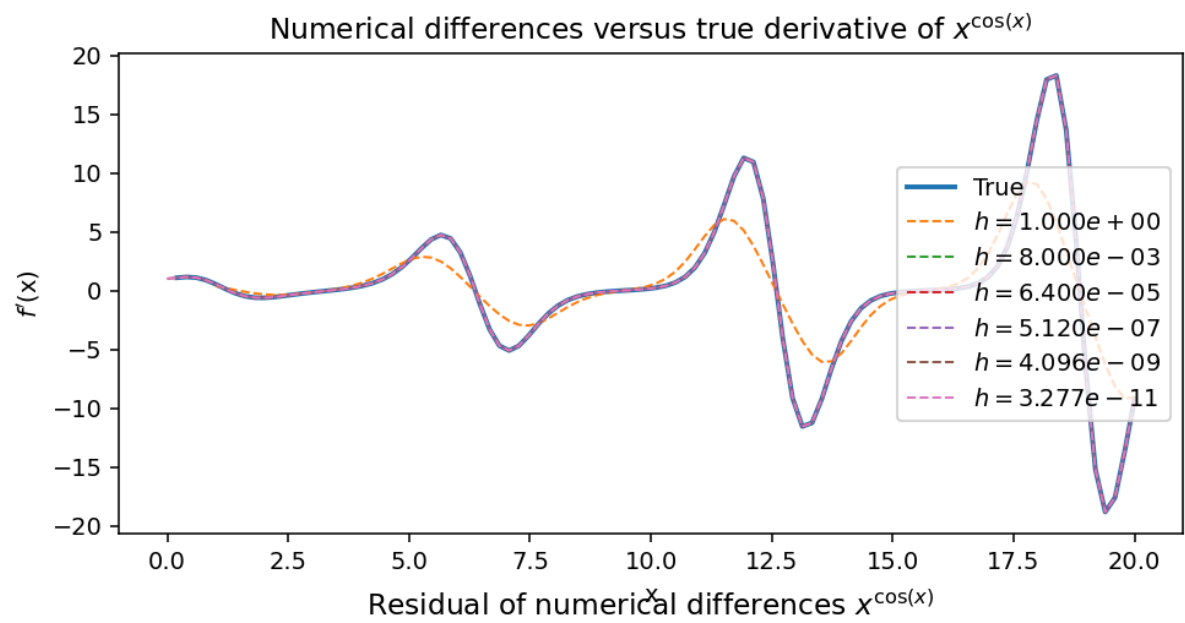
ax.set_yscale("log")
ax.legend(loc='right')
ax.set_title("Residual of numerical differences  $x^{\cos(x)}$ ")
ax.set_xlabel("x")
ax.set_ylabel("Error in computing  $f'(x)$ ")
plt.show()

```

```

/tmp/ipykernel_5100/2637288883.py:5: RuntimeWarning: divide by zero encountered in log
    return x**(np.cos(x)-1) * (np.cos(x)-x*np.log(x)*np.sin(x))
/tmp/ipykernel_5100/2637288883.py:5: RuntimeWarning: invalid value encountered in multiply
    return x**(np.cos(x)-1) * (np.cos(x)-x*np.log(x)*np.sin(x))
/tmp/ipykernel_5100/2637288883.py:2: RuntimeWarning: invalid value encountered in power
    return x**(np.cos(x))

```



autograd



```
In [15]: # uncomment if you want to install this
# !pip install git+git://github.com/HIPS/autograd
```

```
In [16]: # code from: https://github.com/HIPS/autograd
# MIT License: see
# https://github.com/HIPS/autograd/blob/master/license.txt
# note: you will need to install autograd to get this to work!
```

```
import autograd.numpy as np
from autograd import grad, elementwise_grad

# just plain numpy
def tanh(x): # Define a function
    y = np.exp(-x)
    return (1.0 - y) / (1.0 + y)

# compute gradient
grad_tanh = grad(tanh) # Obtain its gradient function
print(grad_tanh(1.0)) # Evaluate the gradient at x = 1.0
```

0.39322386648296376

```
In [17]: # Compare to finite differences
print((tanh(1.0001) - tanh(0.9999)) / 0.0002)
```

0.39322386636453377

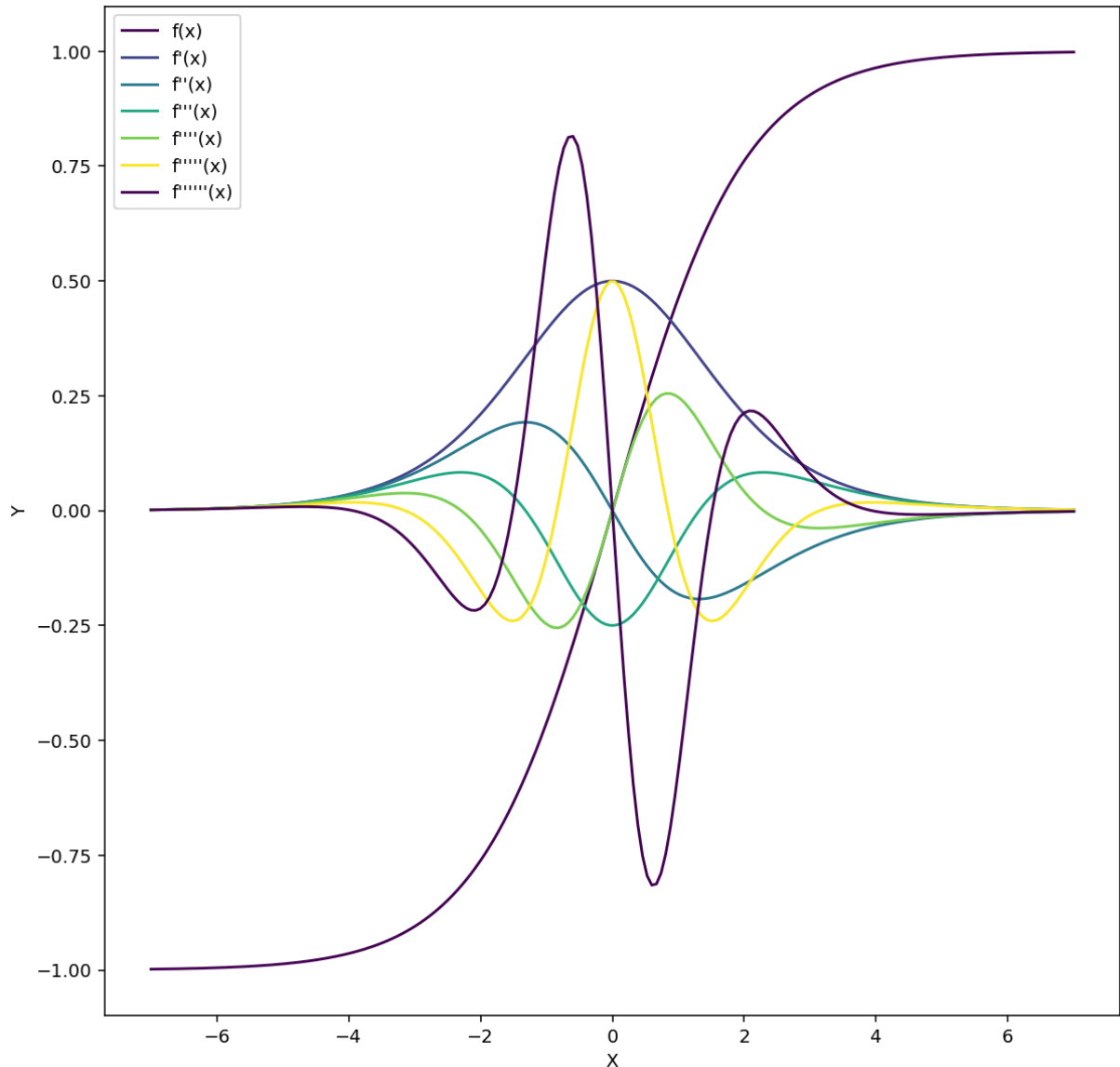
```
In [18]: x = np.linspace(-7, 7, 200)
gr = elementwise_grad
fig = plt.figure(figsize=(10,10))
ax = fig.add_subplot(1,1,1)
ax.plot(x, tanh(x),
        x, gr(tanh)(x),
        derivative
        x, gr(gr(tanh))(x),
        x, gr(gr(gr(tanh)))(x),
        x, gr(gr(gr(gr(tanh))))(x),
        x, gr(gr(gr(gr(gr(tanh))))(x),
        # first
        # second derivative
        # third derivative
        # fourth derivative
        # fifth derivative
```

```

x, gr(gr(gr(gr(gr(gr(tanh))))))(x)) # sixth derivative

ax.set_xlabel("X")
ax.set_ylabel("Y")
ax.legend(["f(x)", "f'(x)", "f''(x)", "f'''(x)", "f''''(x)", "f'''''(x)",
"f''''''(x)"])
plt.show()

```



```

In [19]: # vector -> scalar function

def tanh_sum_sqr(x, k): # Define a function
    return np.tanh(np.sum(x**2)-k)

tanh_sum_sqr(np.array([-0.1, 0.2, 0.6]), 0.5)

```

```

Out[19]: -0.08975778474716013

```

```

In [20]: grad_t_s_s = grad(tanh_sum_sqr) # function -> function

```

```
# differentiates w/respect to first arg. only
grad_t_s_s(np.array([-0.1, 0.2, 0.3]), 0.5)
```

```
Out[20]: array([-0.17616545,  0.35233091,  0.52849636])
```

```
In [21]: # I do not fancy trying to differentiate this by hand
# (actually it's not that hard)
# (just really annoying)
f = np.array([[1, -0.2], [0.8, 1.5]])

def dog(x):
    x = x @ f
    return np.exp(-np.sum(x ** 2, axis=1) / 10.0) - np.exp(
        -np.sum(x ** 2, axis=1) / 2.0
    )

ddog = elementwise_grad(dog) # compute derivative via autograd
# that is *literally* all that is required
```

```
In [22]: ## plot the results
# generate a grid to evaluate the function over
def show_fn(ax, fn, r, n_steps, d=None, contours=20):
    spacing = np.linspace(r[0], r[1], n_steps)
    mx, my = np.meshgrid(spacing, spacing)

    # rearrange the points into a Nx2 array
    pts = np.stack([mx.ravel(), my.ravel()]).T
    if d is None:
        mz = fn(pts).reshape(mx.shape)
    else:
        mz = fn(pts[:, d]).reshape(mx.shape)
    ax.pcolor(mx, my, mz)
    ax.contour(mx, my, mz, colors="k", linewidths=0.3, contours=contours)
    ax.set_aspect(1.0)
```

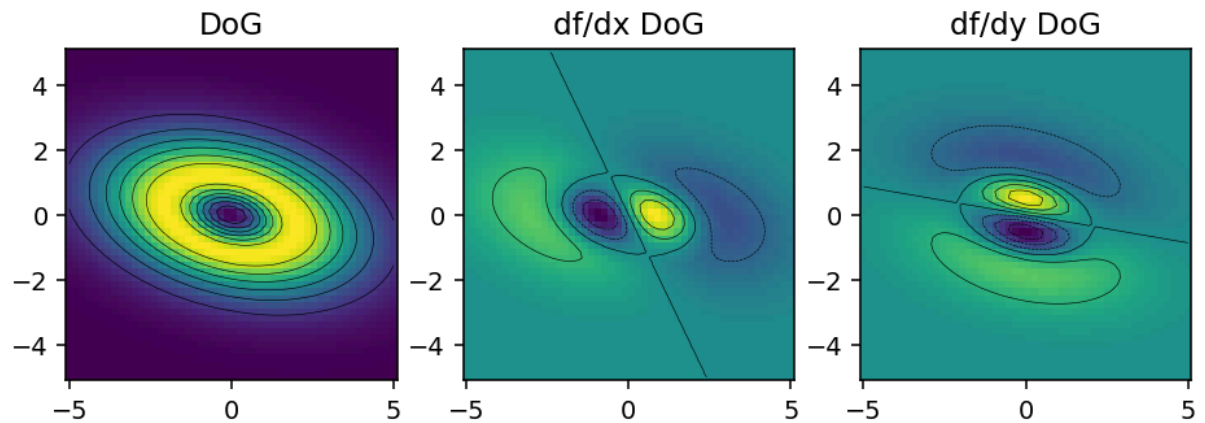
```
In [23]: ## plot the results
# generate a grid to evaluate the function over

# plot the results
fig = plt.figure()
ax = fig.add_subplot(1, 3, 1)
show_fn(ax, dog, [-5, 5], 50)
ax.set_title("DoG")
ax = fig.add_subplot(1, 3, 2)
show_fn(ax, ddog, [-5, 5], 50, d=0)
ax.set_title("df/dx DoG")
ax = fig.add_subplot(1, 3, 3)
show_fn(ax, ddog, [-5, 5], 50, d=1)
ax.set_title("df/dy DoG")
plt.show()
```



```
/tmp/ipykernel_5100/1093179204.py:14: UserWarning: The following kwargs were not used by contour: 'contours'
```

```
ax.contour(mx, my, mz, colors="k", linewidths=0.3, contours=contours)
```

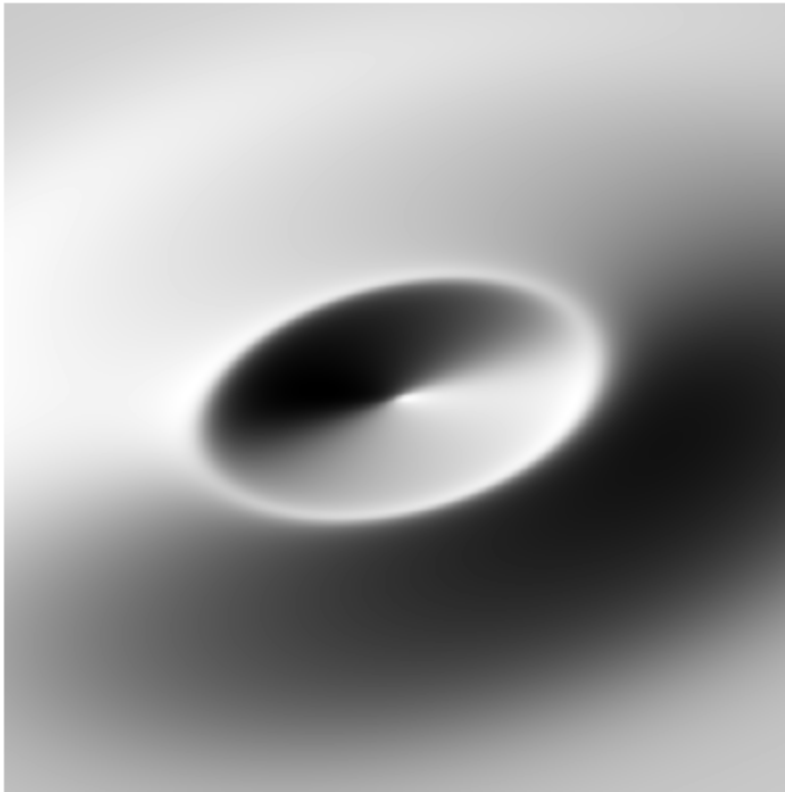


```
In [24]: # You can ignore this if you want
# it just shows how the derivative can be useful
# in visualising a function, by computing lighting
# effects
fig = plt.figure()
ax = fig.add_subplot(1, 2, 1)
spacing = np.linspace(-4, 4, 300)
mx, my = np.meshgrid(spacing, spacing)
# rearrange the points into a Nx2 array
pts = np.stack([mx.ravel(), my.ravel()]).T

# stack up the gradient vectors
dzdx = ddog(pts)[: , 0]
dzdy = ddog(pts)[: , 1]
dzdz = np.ones_like(dzdy) * 0.08

grad_vec = np.stack([dzdx, dzdy, dzdz]).T
# normalise gradient vector
grad_vec = (grad_vec.T / (np.linalg.norm(grad_vec, axis=1))).T
# compute dot product with light vecotr
light_vec = np.array([0.5, 0.3, 0.5])
light = np.sum(grad_vec * light_vec / np.linalg.norm(light_vec), axis=1)

ax.imshow(light.reshape(mx.shape), cmap="gray")
ax.axis("off")
plt.show()
```



```
In [25]: ## generate some random data, with a linear trend
# the linear regression problem; find m and c in the equation y=mx+c
from utils.history import linear_regression_plot

line_x = np.sort(np.random.normal(0, 1, (20,)))
gradient, offset = 2, 1 # these are the "true" parameters that generated
this data

# 2x + 1 and a bit of noise
line_y = (
    gradient * line_x + offset + np.random.normal(0, 0.5 * np.abs(line_x),
line_x.shape)
)

# loss function
def loss(theta):
    # sum of squares
    e = np.sum(((line_x * theta[0] + theta[1]) -
line_y) ** 2)
    return e
```

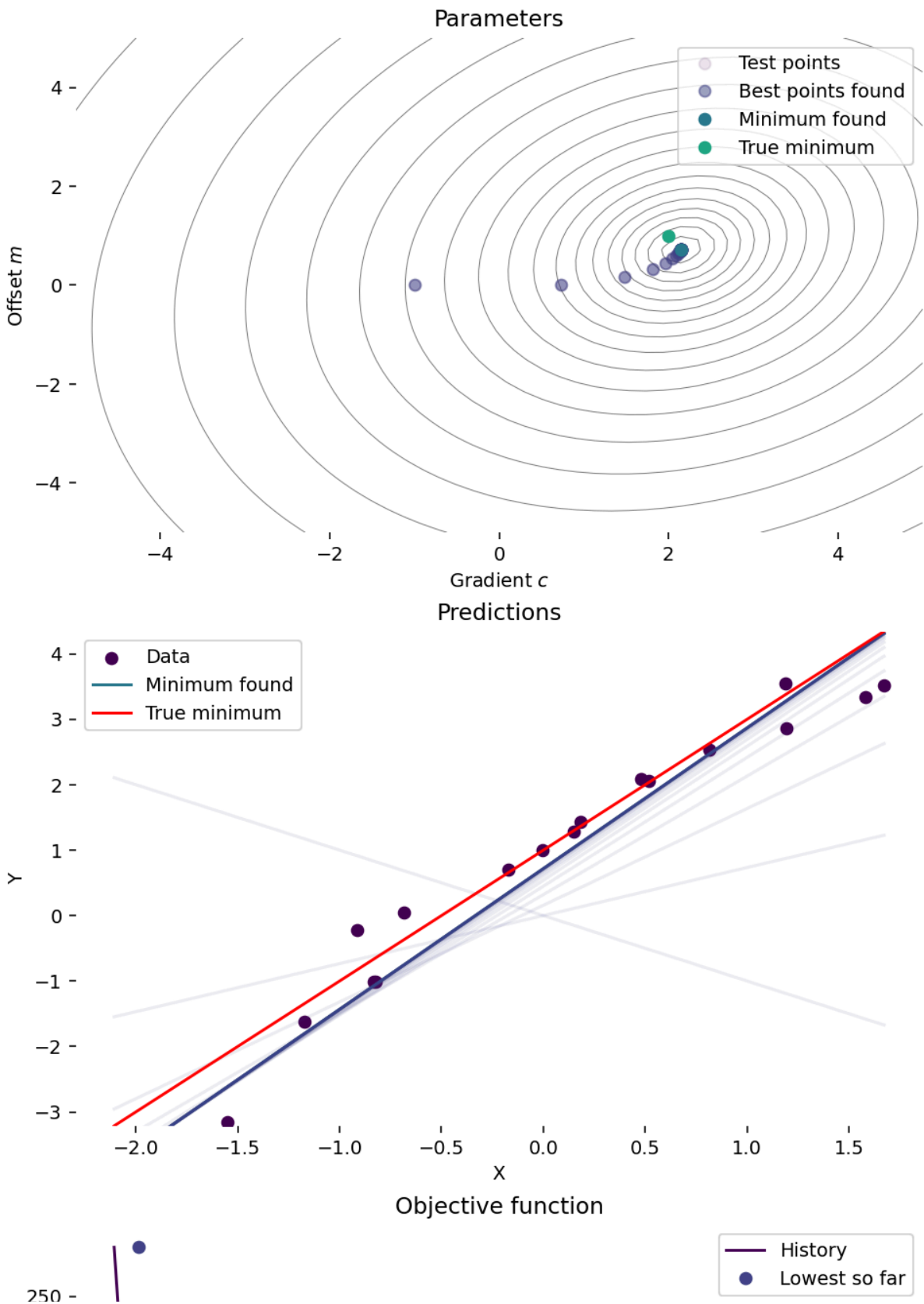
```
In [26]: #### new stuff
# compute derivative (in one step!)
dloss = grad(loss)
```

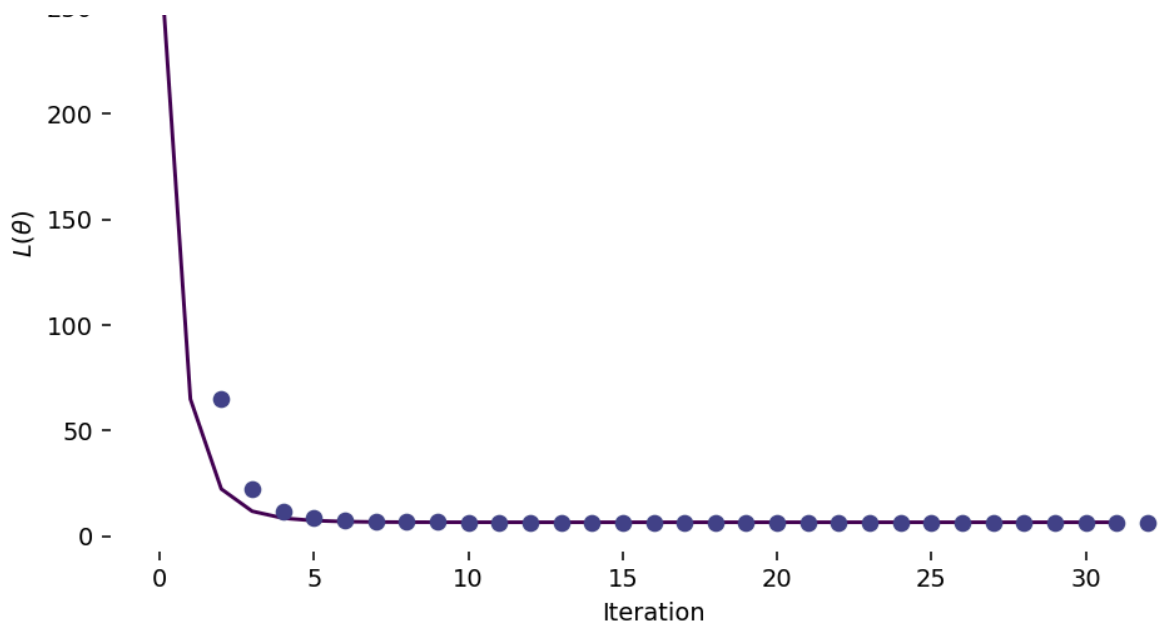
```
# optimise with gradient descent
grad_results = gradient_descent(loss, dloss,
                                np.array([-1.0,0.0]),
                                0.01)
print(grad_results.iters)
```

32

```
In [27]: linear_regression_plot(grad_results, gradient, offset, line_x,
                                line_y, opt_title="Gradient descent")
plt.show()
```

Gradient descent over possible line configurations





Imagedano272CC BY-ND

```
In [28]: fig = plt.figure()
f_ax = fig.add_subplot(1,2,1)
d_ax = fig.add_subplot(1,2,2)

# hard step function
def step(x,t):
    return np.where(x>t, 1, 0)

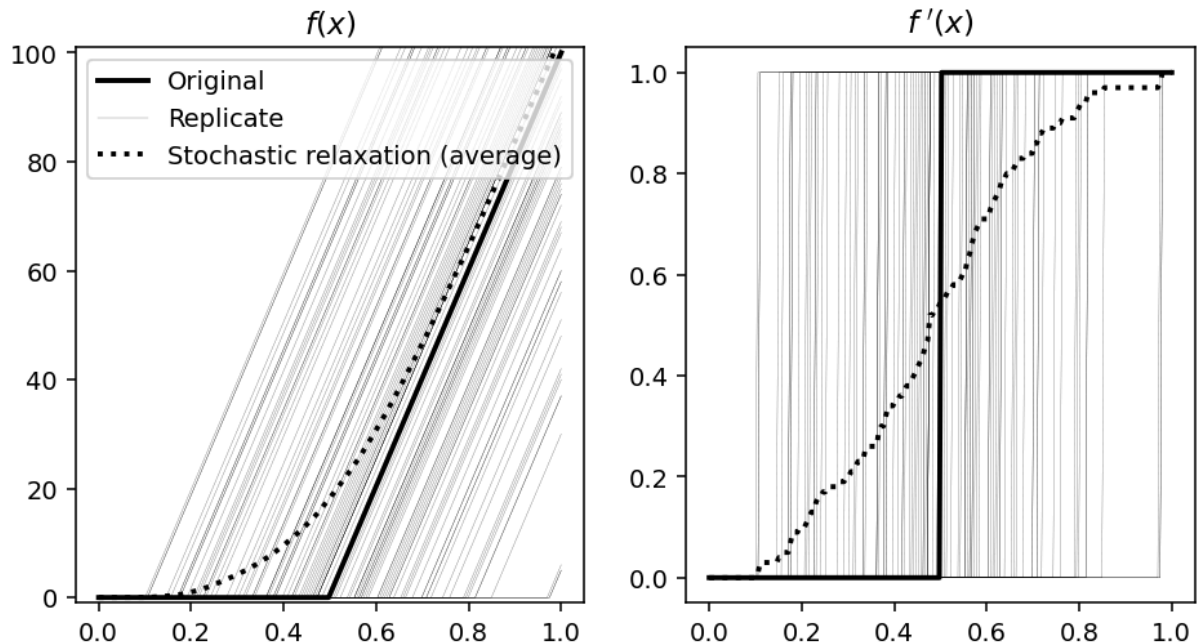
xs = np.linspace(0,1,200)
fn = step(xs, 0.5)
f_ax.plot(xs, np.cumsum(fn), 'k', lw=2, label="Original")
d_ax.plot(xs, fn, 'k', lw=2, label="Original")
f_ax.set_title("$f(x)$")
d_ax.set_title("$f^{\prime}(x)$")

# but if we average it with slight shifts
# we get a function that is smoother
# we don't know the derivative exactly
# but that, it turns out, doesn't matter...
average_step = np.zeros_like(xs)
for i in range(100):
    offset = np.random.normal(0,0.2)
    rnd_step = step(xs, 0.5+offset)
    f_ax.plot(xs, np.cumsum(rnd_step), lw=0.1, c='k')
    d_ax.plot(xs, rnd_step, lw=0.1, c='k')
    average_step += rnd_step
f_ax.plot(xs, np.cumsum(average_step), lw=0.1, c='k', label="Replicate")
```

```

d_ax.plot(xs, rnd_step, lw=0.1, c='k', label="Replicate")
f_ax.set_ylim(-1,101)
d_ax.plot(xs, average_step/100.0, 'k:', lw=2, label="Stochastic relaxation
(average)")
f_ax.plot(xs, np.cumsum(average_step)/100.0, 'k:', lw=2, label="Stochastic
relaxation (average)")
f_ax.legend(loc='upper center')
plt.show()

```



```

In [29]: import autograd.numpy as np
def sgd(L, dL, theta_0, xs, ys, step=0.1, batch_size=10, epochs=10):
    """L: Objective function, in the form L(theta, xs, ys)
       dL: derivative of objective function in form dL(theta, xs, ys)
       theta_0: starting guess
       xs: vector of inputs
       ys: vector of outputs
       step: step size
       batch_size: batchsize
    """
    theta = np.array(theta_0)
    grad = np.zeros_like(theta_0)

    o = History()

    # One epoch is one run through the whole dataset
    for epoch in range(epochs):
        batch = np.arange(len(xs))
        # randomize order
        np.random.shuffle(batch)
        subset_index = 0

```

```

    # iterate over subsets

    ## One batch
    while subset_index + batch_size <= len(xs):

        # accumulate partial gradient
        i,j = subset_index, subset_index+batch_size

        # compute gradient on the random subset
        # accumulating the gradient direction as we go
        grad = dL(theta, xs[batch[i:j]], ys[batch[i:j]]) / (j-i)
        # next batch please!
        subset_index += batch_size

        # make a step
        theta = theta - grad * step
        o.track(theta, L(theta, xs, ys))

    #o.track(theta, L(theta, xs, ys))
    return o.finalise()

```

```

In [30]: # 100000 data points
xs = np.linspace(0,5, 100_000)
ys = xs * 1.0 + 2.0
ys = ys + np.random.normal(0,0.5,xs.shape)

def linear_loss(theta,xs,ys):
    y_prime = theta[0] * xs + theta[1]
    loss = np.sum((y_prime-ys)**2)
    return loss

def linear_dloss(theta,xs,ys):
    return grad(lambda x:linear_loss(x,xs,ys))(theta)

```

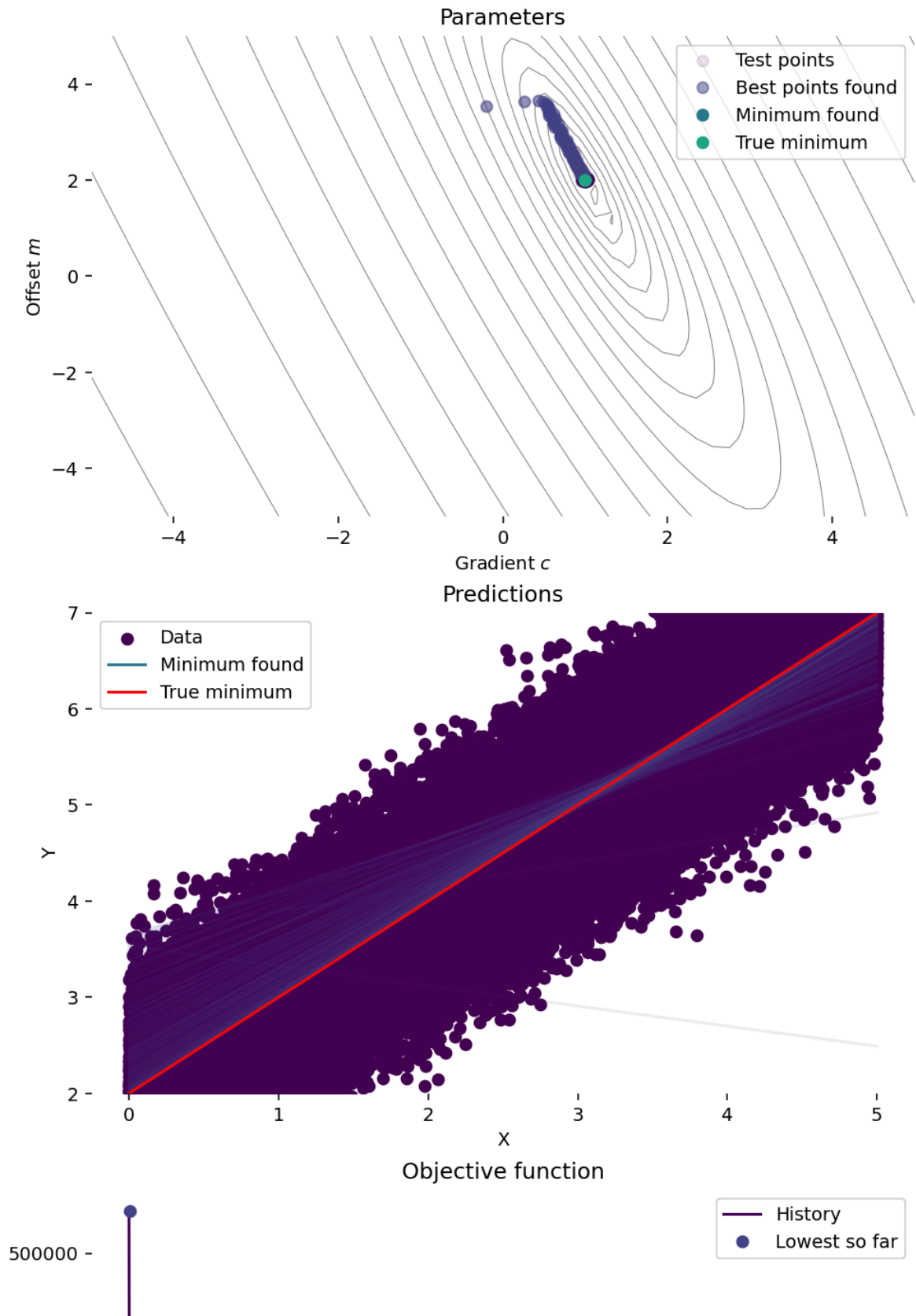
```

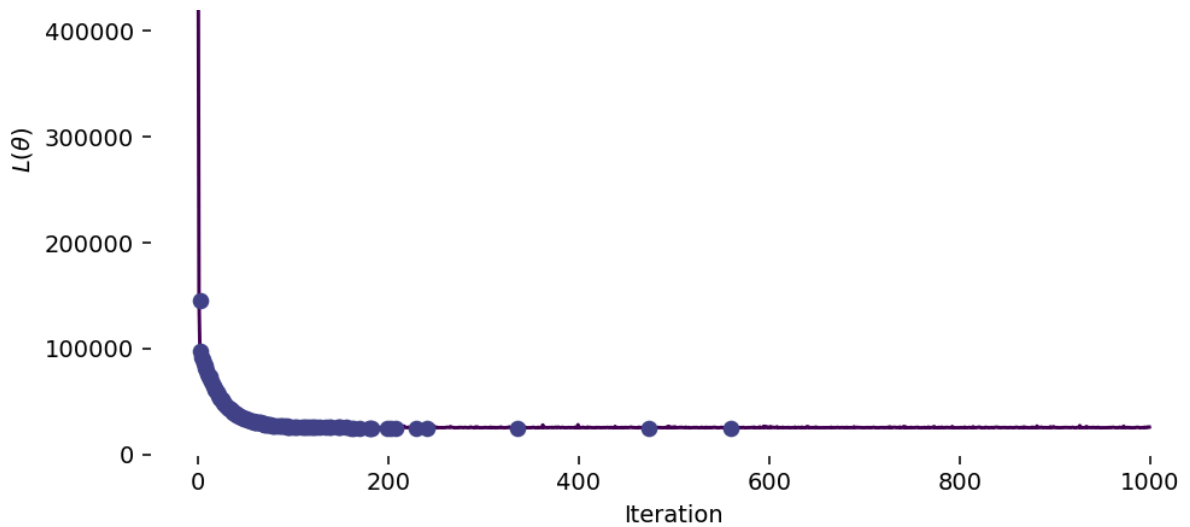
In [31]: res = sgd(linear_loss, grad(linear_loss), [-3.5, 2.5], xs, ys,
                  batch_size=100, epochs=1, step=0.05)

linear_regression_plot(res, 1.0, 2.0, xs, ys, 'SGD')
plt.show()

```

SGD over possible line configurations





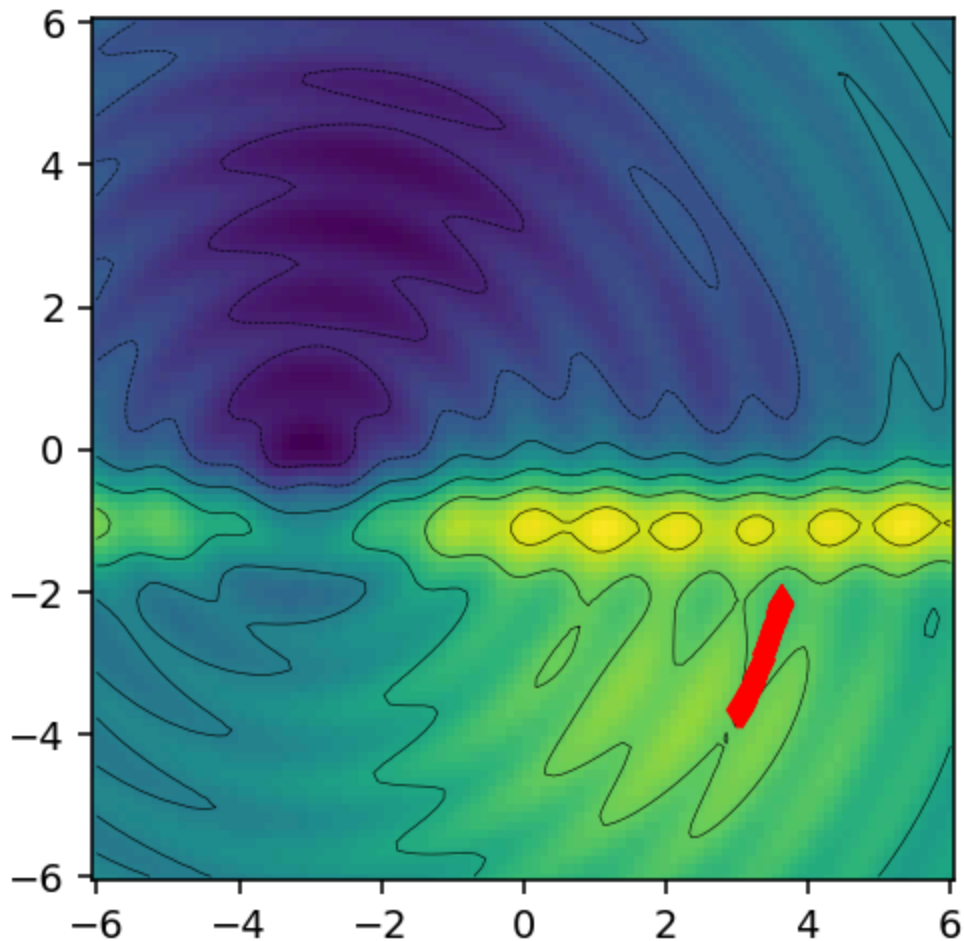
```
In [32]: # this function is crazy
# don't worry about how it is defined
# it is designed to be horrible, not realistic
def bad_function(theta):
    if len(theta.shape)==1:
        theta = theta[None,:]
    ctr = np.array([-2, 3])
    false_ctr = np.array([3,0.5])
    loss = -np.exp(-(np.sum((theta-ctr)**2, axis=1)/20.0))
    loss = loss + np.exp(-(np.sum((theta+ctr)**2, axis=1)/20.0))
    loss = loss + np.cos(np.linalg.norm(theta+false_ctr, ord=2,
axis=1)*6)*0.08
    loss = loss - np.exp(-(np.sum((theta-false_ctr)**2, axis=1)/5.0))*0.75
    loss = loss - np.exp(-(np.sum((theta+false_ctr)**2, axis=1)/2.0))*0.75
    loss = loss + np.exp(-((theta[:,1]+1)**2)*2)
    return loss
```

```
In [33]: # differentiate and do gradient descent
dbad_function = elementwise_grad(bad_function)
res = gradient_descent(bad_function, dbad_function,
np.random.normal(0,4,2), 0.5)

fig = plt.figure()
ax = fig.add_subplot(1,1,1)
show_fn(ax, bad_function, (-6,6), 100, contours=50)
ax.set_aspect(1.0)

plt.plot(res.all_theta[:,0], res.all_theta[:,1], 'rd')
plt.show()
```

```
/tmp/ipykernel_5100/1093179204.py:14: UserWarning: The following kwargs were not used by contour: 'contours'  
ax.contour(mx, my, mz, colors="k", linewidths=0.3, contours=contours)
```



```
In [34]: def gradient_descent_restart_momentum(L, dL, sample_fn, delta,  
                                                alpha=0.9, restarts=20,  
                                                tol=1e-4):  
  
    o = History()  
    # retry a number of times  
    for i in range(restarts):  
        # copy theta_0  
        theta = np.array(sample_fn())  
        o.loss_change = np.inf # force the optimiser to restart  
  
        # while the loss changes  
        vel = dL(theta)  
        while np.abs(o.loss_change)>tol:  
  
            # accumulate velocity "keep on rollin"
```

```

        vel = alpha * vel + delta * dL(theta)
        # make a step along the velocity
        theta -= vel
        o.track(np.array(theta), L(theta))

    # force a break in the plot
    o.track(np.full(theta.shape, np.nan), L(theta))
    return o.finalise()

```

```

In [35]: ### now plot the trace with momentum and restarts

res = gradient_descent_restart_momentum(
    bad_function, dbad_function, lambda=np.random.normal(0,2,2),
    delta=0.2, alpha=0.9)

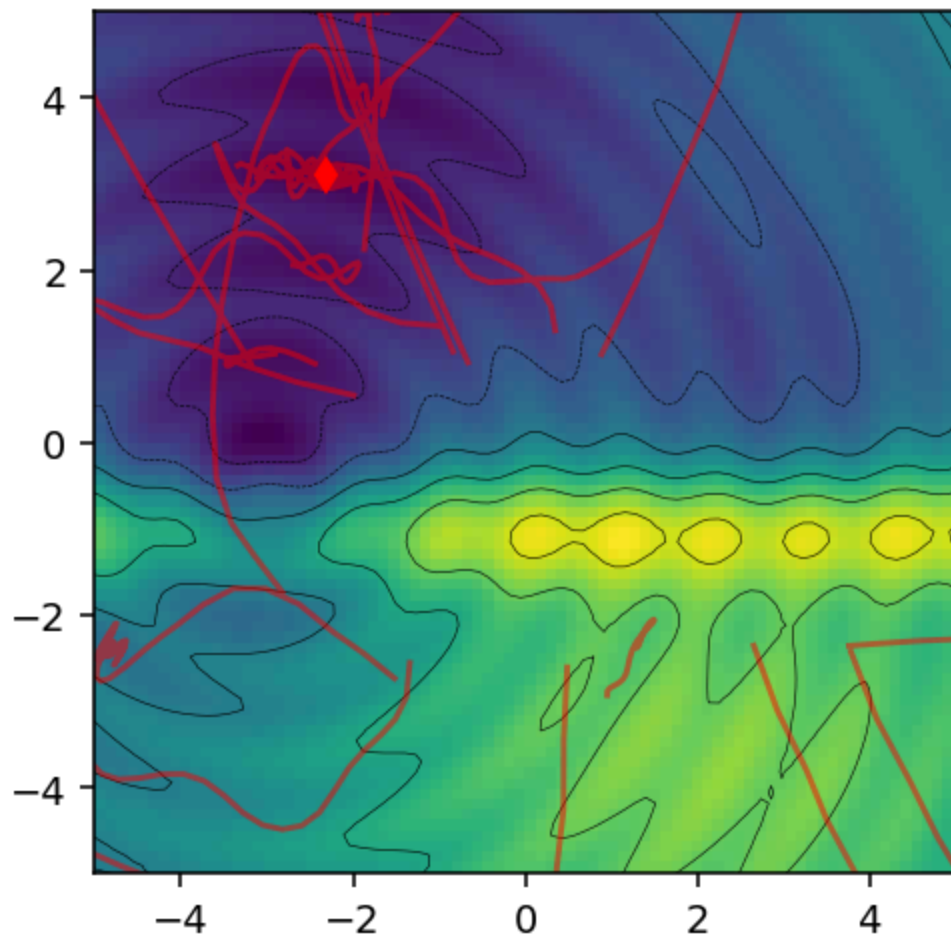
fig = plt.figure()
ax = fig.add_subplot(1,1,1)
show_fn(ax, bad_function, (-6,6), 100, contours=50)
ax.set_aspect(1.0)
ax.set_xlim(-5,5)
ax.set_ylim(-5,5)
ax.plot(res.all_theta[:,0], res.all_theta[:,1], 'r', alpha=0.5)
ax.plot(res.best_theta[0], res.best_theta[1], 'rd')
plt.show()

```

```

/tmp/ipykernel_5100/1093179204.py:14: UserWarning: The following kwargs were
not used by contour: 'contours'
    ax.contour(mx, my, mz, colors="k", linewidths=0.3, contours=contours)

```



```
In [36]: from autograd import grad, jacobian
         from utils.optimisers import random_search, hill_climbing,
```

```

gradient_descent
import scipy.optimize
import autograd.numpy as np

# the linear regression problem; find m and c in the equation y=mx+c
dim = 15
x = np.sort(np.random.normal(0,1,(20,dim)))
A = np.random.normal(0,1,(dim,dim))
b = np.random.normal(0,10,(dim,))

y = (x@A) + b + np.random.normal(0, 0.5*np.abs(x), x.shape)
l = len(A.ravel())
params = l + len(b)
%matplotlib inline
%matplotlib inline

import matplotlib.pyplot as plt

def loss(theta):
    # sum of squares
    Ap = theta[:l].reshape(A.shape)
    bp = theta[l:]

    e = np.sum(((x @ Ap + bp) - y)**2)
    return e

# compute derivative (in one step!)
dloss = grad(loss)

guess = lambda: np.random.normal(0, 1, params)
neighbour = lambda x: x+np.random.normal(0, 0.01, params)

random_res = random_search(loss, guess, 5000)
hill_res = hill_climbing(loss, guess, neighbour, 5000)

gd_res = gradient_descent(loss, dloss, guess(), 1e-3, maxiter=5000)

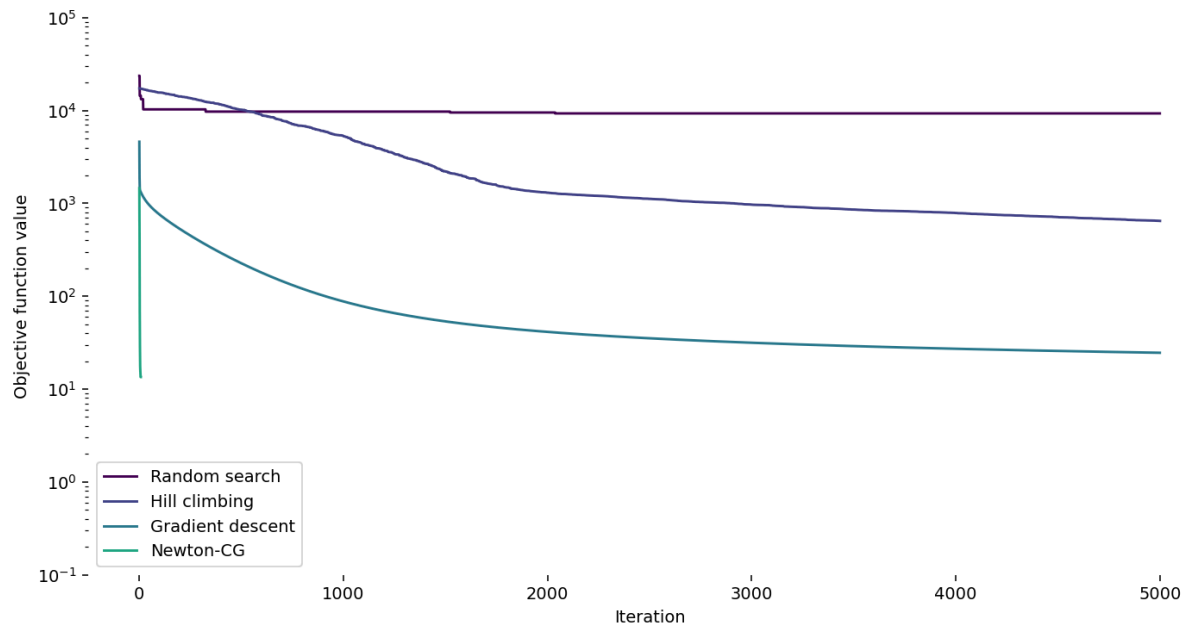
ncg_trace = []
def record_trace(x):
    ncg_trace.append(loss(x))
ncg_res = scipy.optimize.minimize(fun=loss,
                                  x0=guess(),
                                  callback=record_trace,
                                  method='Newton-CG',
                                  jac=grad(loss),
                                  hess=jacobian(grad(loss)))

fig = plt.figure(figsize=(12,6))
ax = fig.add_subplot(1,1,1)
ax.plot(random_res.loss_trace, label="Random search")
ax.plot(hill_res.loss_trace, label="Hill climbing")
ax.plot(gd_res.loss_trace, label="Gradient descent")
ax.plot(ncg_trace, label="Newton-CG")

ax.set_frame_on(False)

```

```
ax.set_yscale("log")
ax.set_ylim(1e-1, 1e5)
ax.legend()
ax.set_xlabel("Iteration")
ax.set_ylabel("Objective function value")
plt.show()
```



float64

```
In [37]: print("{0} bytes of memory".format(1000000 * 1000000 * 8)) # 8 bytes per float
```

8000000000000 bytes of memory

[How machines learn](#)

[What is backpropagation](#)

[Gradient based optimization](#)

[An overview of gradient descent optimization algorithms](#)

[An introduction to algorithms for continuous optimization](#)